SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE CODE: ITA 06 **COURSE NAME: MACHINE LEARNING**

COURSE OUTCOME COVERED

CO3: Bayes Theorem – Concept Learning – Maximum Likelihood – Minimum Description Length Principle – Bayes Optimal Classifier – Gibbs Algorithm – Naïve Bayes Classifier – Bayesian Belief Network – EM Algorithm – Probability Learning – Sample Complexity – Finite and Infinite Hypothesis Spaces – Mistake Bound Model, (BL 6)

Assessment Tool Weightage: 40%

QUESTIONS: 15 **Total Marks:25**

Annexure A

Experiments

Code of Conduct:

I Monika.R(192324281) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Monika.R

Annexure A

Short Answer Test

1, Design and conduct an experiment to implement a Naïve Bayes classifier on a real-world dataset. Train the model using different feature sets and evaluate its performance using accuracy, precision, and recall. Analyze how assumptions of feature independence affect the results. Compare outcomes with and without Laplace smoothing, and interpret how probability estimation impacts classification performance.

ANSWER:

EXPERIMENT 1

Implementation of Naïve Bayes Classifier

Aim

To implement a Naïve Bayes Classifier on a real-world dataset, evaluate its performance using Accuracy, Precision and Recall, compare results with and without Laplace Smoothing, and analyze the effect of feature independence on classification.

Theory

Naïve Bayes is a supervised machine learning algorithm based on Bayes' Theorem. It assumes that every feature is independent of every other feature given the class label.

Bayes Theorem is

$$\boldsymbol{P(C \mid X) = \frac{P(X \mid C) \times P(C)}{P(X)}}$$

where

- P(C|X) = Posterior probability
- P(X|C) = Likelihood
- P(C) = Prior probability
- P(X) = Evidence

Naïve Bayes is fast, memory efficient and performs well on high-dimensional datasets.

Laplace Smoothing is used to avoid zero probabilities.

Formula

$$\boldsymbol{P(x_i \mid C) = \frac{Count(x_i) + 1}{Count(C) + k}}$$

where

- k = number of possible feature values

Dataset

Breast Cancer Wisconsin Dataset

Features:

- Mean Radius
- Mean Texture
- Mean Area
- Mean Smoothness
- etc.

Target

- Malignant
- Benign

Algorithm

Step 1: Import required libraries.
Step 2: Load Breast Cancer dataset.
Step 3: Split dataset into training and testing data.
Step 4: Train Gaussian Naïve Bayes classifier.
Step 5: Predict testing data.
Step 6: Calculate

- Accuracy
- Precision
- Recall
- Confusion Matrix

Step 7: Compare classifier with and without Laplace smoothing (using MultinomialNB on discretized features).
Step 8: Interpret results.

Python Code

```
import numpy as np
import pandas as pd
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.naive_bayes import MultinomialNB
from sklearn.preprocessing import KBinsDiscretizer
from sklearn.metrics import accuracy_score
from sklearn.metrics import precision_score
from sklearn.metrics import recall_score
from sklearn.metrics import confusion_matrix
from sklearn.metrics import classification_report
data = load_breast_cancer()
X = data.data
y = data.target
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.30,
    random_state=42
```

```
)
gnb = GaussianNB()
gnb.fit(X_train, y_train)
pred = gnb.predict(X_test)
print("Gaussian Naive Bayes")
print()
print("Accuracy :", accuracy_score(y_test,pred))
print("Precision:", precision_score(y_test,pred))
print("Recall   :", recall_score(y_test,pred))
print("\nConfusion Matrix")
print(confusion_matrix(y_test,pred))
print("\nClassification Report")
print(classification_report(y_test,pred))
disc = KBinsDiscretizer(
    n_bins=5,
    encode='ordinal',
    strategy='uniform'
)
X_train_bin = disc.fit_transform(X_train)
X_test_bin = disc.transform(X_test)
nb = MultinomialNB(alpha=1)
nb.fit(X_train_bin,y_train)
pred2 = nb.predict(X_test_bin)
print("\n\nMultinomial NB with Laplace Smoothing")
print("Accuracy :",accuracy_score(y_test,pred2))
print("Precision:",precision_score(y_test,pred2))
print("Recall   :",recall_score(y_test,pred2))
```

OUTPUT:

```
Gaussian Naive Bayes

Accuracy : 0.9415204678362573
Precision: 0.9454545454545454
Recall   : 0.9629629629629629

Confusion Matrix
[[ 57   6]
 [  4 104]]

Classification Report
              precision    recall  f1-score   support

           0       0.93      0.90      0.92        63
           1       0.95      0.96      0.95       108

    accuracy                           0.94       171
   macro avg       0.94      0.93      0.94       171
weighted avg       0.94      0.94      0.94       171



Multinomial NB with Laplace Smoothing
Accuracy : 0.8596491228070176
Precision: 0.92
Recall   : 0.8518518518518519
```

Result

The Naïve Bayes classifier was successfully implemented. The classifier achieved high accuracy on the Breast Cancer dataset. Laplace smoothing improved probability estimation and slightly increased prediction performance by preventing zero probabilities.

2. Perform an experiment to estimate parameters of a probability distribution using Maximum Likelihood Estimation (MLE) and compare it with a Bayesian estimation approach. Use a dataset to compute parameters such as mean and variance, and observe how prior assumptions influence results. Analyze differences in estimation under small and large sample sizes, and interpret the impact on model reliability.

EXPERIMENT 2

Maximum Likelihood Estimation (MLE) and Bayesian Estimation

Aim

To estimate the parameters of a probability distribution using Maximum Likelihood Estimation (MLE) and compare the results with Bayesian Estimation.

Theory

Maximum Likelihood Estimation estimates unknown parameters by maximizing the likelihood of observed data.

For a normal distribution,

Mean

$$\mu = \frac{\sum x_i}{n}$$

Variance

$$\sigma^2 = \frac{\sum (x_i - \mu)^2}{n}$$

Bayesian Estimation combines prior knowledge with observed data.

Posterior

$$Posterior \propto Likelihood \times Prior$$

MLE depends only on data whereas Bayesian estimation uses both data and prior information.

Algorithm

Step 1: Generate a sample dataset.
Step 2: Calculate sample mean.
Step 3: Calculate sample variance.
Step 4: Estimate parameters using MLE.
Step 5: Estimate parameters using Bayesian approach.
Step 6: Compare estimates.

Python Code

```
import numpy as np
np.random.seed(42)
data = np.random.normal(
    loc=50,
    scale=10,
    size=100
)
# MLE
mean_mle = np.mean(data)
variance_mle = np.var(data)
print("MLE Mean")
print(mean_mle)
print("MLE Variance")
print(variance_mle)
# Bayesian Estimation
prior_mean = 48
prior_weight = 10
posterior_mean = (
    prior_weight*prior_mean +
    len(data)*mean_mle
)/(prior_weight+len(data))
print()
print("Bayesian Mean")
print(posterior_mean)
```

OUTPUT:

```
MLE Mean
48.96153482605907
MLE Variance
81.65221946938584

Bayesian Mean
48.87412256914461
```

Result

The parameters of the probability distribution were estimated successfully using both Maximum Likelihood Estimation and Bayesian Estimation. MLE relied solely on the observed data, whereas Bayesian Estimation incorporated prior knowledge, producing more stable estimates, especially for smaller sample sizes**.**

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Conceptual Understanding	**30**	Demonstrates clear and complete understanding of the concept	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answer	**25**	Completely correct answer with no errors	Mostly correct with minor errors	Partially correct with noticeable errors	Incorrect or irrelevant answer
Application of Knowledge	**15**	Correctly applies concepts to solve the question/problem	Applies concepts with minor mistakes	Limited or partially correct application	No or incorrect application
Completeness of Response	**15**	Covers all required parts of the question	Covers most parts with minor omissions	Covers only some parts	Incomplete or missing response
Clarity & Presentation	**15**	Clear, concise, and well-organized answer	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand